JavaScript Error object

ITI - Assiut Branch

Eng. Sara Ali

Error Object

- Whenever an error occurs, an instance of the Error object is created to describe the error.

- **Error objects can be created in 2 ways:**
  - Explicitly:

```
var newErrorObj = new Error();
```

  - Implicitly:

    thrown using the throw statement

- **Error Properties**

Property	Description
fileName	URI of the file containing the script throwing the error
lineNumber	Source code line number of error
message	Plain-language description of error (ECMA)
name	Error type (ECMA)
number	Microsoft proprietary error number

Error Object (Cont.)

- Error Name Values

  Six different values can be returned by the error name property:

  var e = new Error();

Error Name	Description
EvalError	An error has occurred in the eval() function
RangeError	A number "out of range" has occurred
ReferenceError	An illegal reference has occurred
SyntaxError	A syntax error has occurred
TypeError	A type error has occurred
URIError	An error in encodeURI() has occurred

Using **instanceOf** when catching the error lets you know if the error is one of these built-in types.

Error Object (Cont.)

- Example:

```
var e = new EvalError('jaavcsritp is _not_ how you spell it');
document.write(e.name); //EvalError
document.write(e. message); // jaavcsritp is _not_ how you spell it
```

- Error Handling:
  1. try…catch
  2. onerror

1- try…catch

- The try...catch statement allows you to test a block of code for errors.
- The try block contains the code to be run.
- The catch block contains the code to be executed if an error occurs.

Error Object (Cont.)

- **Error Handling (cont.):**

  1- try…catch

  - The try...catch statement allows you to test a block of code for errors.
  - The try block contains the code to be run.
  - The catch block contains the code to be executed if an error occurs.

```
try
{
        //Run some code here
 }
catch(err)
{
        //Handle errors here
}
```

Implicitly Created Error object "err"

Error Object (Cont.)

- **Error Handling (cont.):**

  1- try…catch

  - If you have any functionality that needs to be processed regardless of success or failure, you can include this in the finally block (will executed even if there break, throw, or return statements in catch block.

```
try
{
        //Run some code here
 }
catch(err)
{
        //Handle errors here
}
finally
{
        //Here code that will be executed on both cases
}
```

Error Object (Cont.)

- **Error Handling (cont.):**

  1- onerror

  - The old standard solution to catch errors in a web page.
  - The onerror event is fired whenever there is a script error in the page.
  - onerror event can be used to retrieve additional information about the error.

```
onerror=handleErr
function handleErr(msg,url,l)
{
        //Handle the error here
                return true;
}
```

msg : Contains the message explaining why the error occurred.

url : Contains the url of the page with the errorscript .

l : Contains the line number where the error occurred

Error Object (Cont.)

- **Throw Error**
  - The throw statement allows you to create an exception.
  - Syntax:

```
throw(exception)
```

  - The exception can be a string, integer, Boolean or an object.

```
try{
        if(x<100)
                throw "less100"
        else if(x>200)
                throw "more200"
    }
catch(er){
                if(er=="less100")
                    alert("Error! The value is too low")
                if(er == "more200")
                    alert("Error! The value is too high")
            }
```

JavaScript

THANK YOU